

RAG Pipeline - Agentic RAG: Self-Reflective RAG (Self-RAG)

1. Definition

Self-RAG (Self-Reflective Retrieval-Augmented Generation) is an advanced agentic architecture that teaches a Large Language Model (LLM) to critique its own retrieval and generation processes.

Unlike standard RAG (which retrieves blindly) or CRAG (which evaluates documents before generation), Self-RAG evaluates the **generated answer itself**. It uses special "Reflection Tokens" to control the model's behavior, ensuring that the final output is factually grounded and useful.

2. Core Mechanism: Reflection Tokens

The defining feature of Self-RAG is the use of four specific "tokens" (or classification decisions) that the model makes during the generation process. In a LangChain implementation, these are typically handled by separate "Grader" chains.

Token / Decision	Meaning	Question It Asks
[Retrieve]	Retrieval Necessity	"Does this query actually <i>need</i> external data, or can I answer it from my own memory?"
[IsREL]	Relevance Check	"Is this retrieved document actually relevant to the user's query?" (Similar to CRAG).
[IsSUP]	Support/Faithfulness	"Is the sentence I just generated fully supported by the retrieved evidence?" (Prevents hallucinations).
[IsUSE]	Utility	"Does this answer actually help the user?"

3. The Workflow (Step-by-Step)

Self-RAG operates in a loop of **Generation**, **Critique**, and **Selection**.

Step 1: Retrieval Decision ([Retrieve])

The model analyzes the user's query first.

- *If simple* (e.g., "Write a poem"): Skip retrieval.
- *If factual* (e.g., "Who is the CEO of Apple?"): Trigger retrieval.

Step 2: Generation & Parallel Grading

The system retrieves documents and starts generating an answer. Uniquely, it often generates **multiple candidate segments** and grades them simultaneously.

Step 3: Self-Critique ([IsSUP] & [IsUSE])

For every generated segment (sentence or paragraph), the model runs a self-check:

1. **Check Faithfulness (IsSUP)**: It compares the generated text against the retrieved documents.
 - *If unsupported*: It discards the segment and regenerates it to be faithful to the text.
2. **Check Utility (IsUSE)**: It compares the generated text against the user's question.
 - *If not useful*: It refines the answer to be more direct.

Step 4: Selection

The system picks the "best" segment that passed all critiques and appends it to the final answer. This ensures the final output is a "gold standard" response.

4. Difference from CRAG

It is easy to confuse CRAG and Self-RAG, as both use "Graders."

- **CRAG (Corrective RAG)**: Focuses on the **Input**.
 - *Action*: Grades the *documents*.
 - *Goal*: Fix the retrieval (e.g., Search Web if docs are bad) *before* generation starts.
- **Self-RAG**: Focuses on the **Output**.
 - *Action*: Grades the *generation* (the answer).
 - *Goal*: Fix the hallucination *during* or *after* generation.

Note: In advanced production systems, you often combine both (CRAG for retrieval, Self-RAG for generation).

5. Key LangChain Tools

Component	Tool / Implementation
Orchestration	LangGraph : Essential for the loops. You define nodes for retrieve, generate, grade_documents, and grade_generation.
Reflection	ChatPromptTemplate (LLM-as-a-Judge) : Since we often don't have the specialized "Self-RAG" fine-tuned model, we implement the "tokens" as prompt-based graders.
* <i>Prompt</i> : "You are a grader. Does the following sentence contain information not present in the context? Answer 'Yes' or 'No'."	
Structured Output	PydanticOutputParser : Ensures the graders return strict boolean or score-based outputs that the graph logic can act on.